

Repository Audit Report

Repository	dipak2-8/github-code-audit-agent
Scan	Full Repository Scan
Risk Level	LOW

Repository Audit Report

dipak2-8/github-code-audit-agent

Repository Overview

- Total security issues found: 5
- Total quality issues found: 11
- GitHub Actions vulnerabilities: 0
- Open PRs analyzed: 0

Repository Issues

Security Issues

- 1) app/slack_client.py — `send\_to\_slack` — line 72
- Full function code

```
def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")
```

```

if not slack_token or not channel_id:
    print(" Slack not configured – skipping notification")
    return

# read markdown for summary
try:
    with open(report_path.replace(".pdf", ".md"), "r") as f:
        report_content = f.read()
except:
    report_content = "Report file not found."

# extract top 3 issues
lines = report_content.split("\n")
issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
top_issues = "\n".join(issues) if issues else "See full report."

# step 1 – send summary message
message_payload = {
    "channel": channel_id,
    "blocks": [
        {
            "type": "header",
            "text": {
                "type": "plain_text",
                "text": f"PR Audit Report - {repo_name}"
            }
        },
        {
            "type": "section",
            "fields": [
                {
                    "type": "mrkdwn",
                    "text": f"*Repository:* \n`{repo_name}`"
                },
                {
                    "type": "mrkdwn",
                    "text": f"*PR Number:* \n#{pr_number}"
                }
            ]
        }
    ]
}

```

```

{
    "type": "mrkdwn",
    "text": f"*Risk Level:*\n{risk_level}"
}
],
{
    "type": "section",
    "text": {
        "type": "mrkdwn",
        "text": f"*Top Issues Found:*\n{top_issues}"
    }
},
{
    "type": "section",
    "text": {
        "type": "mrkdwn",
        "text": "Full PDF report attached below."
    }
}
]
}

msg_response = requests.post(
    "https://slack.com/api/chat.postMessage",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json=message_payload
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

```

```
# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",
        headers={"Authorization": f"Bearer {slack_token}"},
        data={
            "filename": filename,
            "length": len(file_data)
        }
    )

    url_data = url_response.json()
    if not url_data.get("ok"):
        print(f"File upload failed: {url_data.get('error')}")
        return

    upload_url = url_data["upload_url"]
    file_id = url_data["file_id"]

    # step 2b – upload file content to URL
    requests.post(
        upload_url,
        data=file_data,
        headers={"Content-Type": "application/octet-stream"}
    )

    # step 2c – complete upload and share to channel
    complete_response = requests.post(
        "https://slack.com/api/files.completeUploadExternal",
        headers={
            "Authorization": f"Bearer {slack_token}",
            "Content-Type": "application/json"
        }
    )
```

```

},
json={
    "files": [{"id": file_id}],
    "channel_id": channel_id,
    "initial_comment": f"Complete audit report for PR #{pr_number}"
}
)

if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")

```

- **Why it is dangerous**

- The `requests.post(...)` call at line 72 has no timeout, so the process can hang indefinitely on Slack API failures or network stalls.

- **Fixed version of the full function**

```

def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")

    if not slack_token or not channel_id:
        print(" Slack not configured – skipping notification")
        return

    # read markdown for summary
    try:
        with open(report_path.replace(".pdf", ".md"), "r") as f:
            report_content = f.read()
    except:
        report_content = "Report file not found."

    # extract top 3 issues

```

```

lines = report_content.split("\n")
issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
top_issues = "\n".join(issues) if issues else "See full report."

# step 1 - send summary message
message_payload = {
    "channel": channel_id,
    "blocks": [
        {
            "type": "header",
            "text": {
                "type": "plain_text",
                "text": f"PR Audit Report - {repo_name}"
            }
        },
        {
            "type": "section",
            "fields": [
                {
                    "type": "mrkdown",
                    "text": f"*Repository:* \n`{repo_name}`"
                },
                {
                    "type": "mrkdown",
                    "text": f"*PR Number:* \n#{pr_number}"
                },
                {
                    "type": "mrkdown",
                    "text": f"*Risk Level:* \n{risk_level}"
                }
            ]
        },
        {
            "type": "section",
            "text": {
                "type": "mrkdown",
                "text": f"*Top Issues Found:* \n{top_issues}"
            }
        }
    ]
}

```

```

    },
    {
        "type": "section",
        "text": {
            "type": "mrkdwn",
            "text": "Full PDF report attached below."
        }
    }
]
}

msg_response = requests.post(
    "https://slack.com/api/chat.postMessage",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json=message_payload,
    timeout=30
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",
        headers={"Authorization": f"Bearer {slack_token}"},

```

```

data={
    "filename": filename,
    "length": len(file_data)
},
timeout=30
)

url_data = url_response.json()
if not url_data.get("ok"):
    print(f" File upload failed: {url_data.get('error')}")
    return

upload_url = url_data["upload_url"]
file_id = url_data["file_id"]

# step 2b – upload file content to URL
requests.post(
    upload_url,
    data=file_data,
    headers={"Content-Type": "application/octet-stream"},
    timeout=30
)

# step 2c – complete upload and share to channel
complete_response = requests.post(
    "https://slack.com/api/files.completeUploadExternal",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json={
        "files": [{"id": file_id}],
        "channel_id": channel_id,
        "initial_comment": f"Complete audit report for PR #{pr_number}"
    },
    timeout=30
)

```

```

if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")

```

- **Documentation link**

- https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout.html

2) app/slack_client.py — `send\_to\_slack` — line 95

- **Full function code**

```

def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")

    if not slack_token or not channel_id:
        print(" Slack not configured – skipping notification")
        return

    # read markdown for summary
    try:
        with open(report_path.replace(".pdf", ".md"), "r") as f:
            report_content = f.read()
    except:
        report_content = "Report file not found."

    # extract top 3 issues
    lines = report_content.split("\n")
    issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
    top_issues = "\n".join(issues) if issues else "See full report."

    # step 1 – send summary message
    message_payload = {
        "channel": channel_id,

```

```
"blocks": [  
  {  
    "type": "header",  
    "text": {  
      "type": "plain_text",  
      "text": f"PR Audit Report - {repo_name}"  
    }  
  },  
  {  
    "type": "section",  
    "fields": [  
      {  
        "type": "mrkdwn",  
        "text": f"*Repository:*\n`{repo_name}`"  
      },  
      {  
        "type": "mrkdwn",  
        "text": f"*PR Number:*\n#{pr_number}"  
      },  
      {  
        "type": "mrkdwn",  
        "text": f"*Risk Level:*\n{risk_level}"  
      }  
    ]  
  },  
  {  
    "type": "section",  
    "text": {  
      "type": "mrkdwn",  
      "text": f"*Top Issues Found:*\n{top_issues}"  
    }  
  },  
  {  
    "type": "section",  
    "text": {  
      "type": "mrkdwn",  
      "text": "Full PDF report attached below."  
    }  
  }  
]
```

```

}
]
}

msg_response = requests.post(
    "https://slack.com/api/chat.postMessage",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json=message_payload
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",
        headers={"Authorization": f"Bearer {slack_token}"},
        data={
            "filename": filename,
            "length": len(file_data)
        }
    )

    url_data = url_response.json()
    if not url_data.get("ok"):

```

```

print(f" File upload failed: {url_data.get('error')}")
return

upload_url = url_data["upload_url"]
file_id = url_data["file_id"]

# step 2b – upload file content to URL
requests.post(
    upload_url,
    data=file_data,
    headers={"Content-Type": "application/octet-stream"}
)

# step 2c – complete upload and share to channel
complete_response = requests.post(
    "https://slack.com/api/files.completeUploadExternal",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json={
        "files": [{"id": file_id}],
        "channel_id": channel_id,
        "initial_comment": f"Complete audit report for PR #{pr_number}"
    }
)

if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")

```

- **Why it is dangerous**

- The `requests.post(...)` call at line 95 has no timeout, so the process can hang indefinitely while waiting for Slack to return an upload URL.

- Fixed version of the full function

```
def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")

    if not slack_token or not channel_id:
        print(" Slack not configured – skipping notification")
        return

    # read markdown for summary
    try:
        with open(report_path.replace(".pdf", ".md"), "r") as f:
            report_content = f.read()
    except:
        report_content = "Report file not found."

    # extract top 3 issues
    lines = report_content.split("\n")
    issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
    top_issues = "\n".join(issues) if issues else "See full report."

    # step 1 – send summary message
    message_payload = {
        "channel": channel_id,
        "blocks": [
            {
                "type": "header",
                "text": {
                    "type": "plain_text",
                    "text": f"PR Audit Report - {repo_name}"
                }
            },
            {
                "type": "section",
                "fields": [
```

```

{
  "type": "mrkdwn",
  "text": f"*Repository:* \n`{repo_name}`"
},
{
  "type": "mrkdwn",
  "text": f"*PR Number:* \n#{pr_number}"
},
{
  "type": "mrkdwn",
  "text": f"*Risk Level:* \n{risk_level}"
}
]
},
{
  "type": "section",
  "text": {
    "type": "mrkdwn",
    "text": f"*Top Issues Found:* \n{top_issues}"
  }
},
{
  "type": "section",
  "text": {
    "type": "mrkdwn",
    "text": "Full PDF report attached below."
  }
}
]
}

```

```

msg_response = requests.post(
  "https://slack.com/api/chat.postMessage",
  headers={
    "Authorization": f"Bearer {slack_token}",
    "Content-Type": "application/json"
  },
  json=message_payload,

```

```

timeout=30
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",
        headers={"Authorization": f"Bearer {slack_token}"},
        data={
            "filename": filename,
            "length": len(file_data)
        },
        timeout=30
    )

    url_data = url_response.json()
    if not url_data.get("ok"):
        print(f" File upload failed: {url_data.get('error')}")
        return

    upload_url = url_data["upload_url"]
    file_id = url_data["file_id"]

    # step 2b – upload file content to URL
    requests.post(
        upload_url,

```

```

data=file_data,
headers={"Content-Type": "application/octet-stream"},
timeout=30
)

# step 2c – complete upload and share to channel
complete_response = requests.post(
    "https://slack.com/api/files.completeUploadExternal",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json={
        "files": [{"id": file_id}],
        "channel_id": channel_id,
        "initial_comment": f"Complete audit report for PR #{pr_number}"
    },
    timeout=30
)

if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")

```

- **Documentation link**

- https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout.html

3) app/slack_client.py — `send\_to\_slack` — line 113

- **Full function code**

```

def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")

```

```

if not slack_token or not channel_id:
    print(" Slack not configured – skipping notification")
    return

# read markdown for summary
try:
    with open(report_path.replace(".pdf", ".md"), "r") as f:
        report_content = f.read()
except:
    report_content = "Report file not found."

# extract top 3 issues
lines = report_content.split("\n")
issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
top_issues = "\n".join(issues) if issues else "See full report."

# step 1 – send summary message
message_payload = {
    "channel": channel_id,
    "blocks": [
        {
            "type": "header",
            "text": {
                "type": "plain_text",
                "text": f"PR Audit Report - {repo_name}"
            }
        },
        {
            "type": "section",
            "fields": [
                {
                    "type": "mrkdwn",
                    "text": f"*Repository:* \n`{repo_name}`"
                },
                {
                    "type": "mrkdwn",
                    "text": f"*PR Number:* \n#{pr_number}"
                }
            ]
        }
    ]
}

```

```

},
{
    "type": "mrkdwn",
    "text": f"*Risk Level:*\n{risk_level}"
}
]
},
{
    "type": "section",
    "text": {
        "type": "mrkdwn",
        "text": f"*Top Issues Found:*\n{top_issues}"
    }
},
{
    "type": "section",
    "text": {
        "type": "mrkdwn",
        "text": "Full PDF report attached below."
    }
}
]
}

msg_response = requests.post(
    "https://slack.com/api/chat.postMessage",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json=message_payload
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

```

```

# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",
        headers={"Authorization": f"Bearer {slack_token}"},
        data={
            "filename": filename,
            "length": len(file_data)
        }
    )

    url_data = url_response.json()
    if not url_data.get("ok"):
        print(f" File upload failed: {url_data.get('error')}")
        return

    upload_url = url_data["upload_url"]
    file_id = url_data["file_id"]

    # step 2b – upload file content to URL
    requests.post(
        upload_url,
        data=file_data,
        headers={"Content-Type": "application/octet-stream"}
    )

    # step 2c – complete upload and share to channel
    complete_response = requests.post(
        "https://slack.com/api/files.completeUploadExternal",
        headers={
            "Authorization": f"Bearer {slack_token}",

```

```

"Content-Type": "application/json"
},
json={
    "files": [{"id": file_id}],
    "channel_id": channel_id,
    "initial_comment": f"Complete audit report for PR #{pr_number}"
}
)

if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")

```

- **Why it is dangerous**

- The `requests.post(...)` call at line 113 has no timeout, so the file upload to Slack can block indefinitely.

- **Fixed version of the full function**

```

def send_to_slack(repo_name, pr_number, report_path, risk_level="UNKNOWN"):
    """Send summary message + upload full report PDF to Slack."""

    slack_token = os.getenv("SLACK_BOT_TOKEN")
    channel_id = os.getenv("SLACK_CHANNEL_ID")

    if not slack_token or not channel_id:
        print(" Slack not configured – skipping notification")
        return

    # read markdown for summary
    try:
        with open(report_path.replace(".pdf", ".md"), "r") as f:
            report_content = f.read()
    except:
        report_content = "Report file not found."

```

```

# extract top 3 issues

lines = report_content.split("\n")
issues = [l for l in lines if "Function:" in l or "### Issue" in l][:3]
top_issues = "\n".join(issues) if issues else "See full report."

# step 1 – send summary message

message_payload = {
    "channel": channel_id,
    "blocks": [
        {
            "type": "header",
            "text": {
                "type": "plain_text",
                "text": f"PR Audit Report - {repo_name}"
            }
        },
        {
            "type": "section",
            "fields": [
                {
                    "type": "mrkdwn",
                    "text": f"*Repository:* \n`{repo_name}`"
                },
                {
                    "type": "mrkdwn",
                    "text": f"*PR Number:* \n#{pr_number}"
                },
                {
                    "type": "mrkdwn",
                    "text": f"*Risk Level:* \n{risk_level}"
                }
            ]
        },
        {
            "type": "section",
            "text": {
                "type": "mrkdwn",
                "text": f"*Top Issues Found:* \n{top_issues}"
            }
        }
    ]
}

```

```

    }
    },
    {
        "type": "section",
        "text": {
            "type": "mrkdwn",
            "text": "Full PDF report attached below."
        }
    }
]
}

msg_response = requests.post(
    "https://slack.com/api/chat.postMessage",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json=message_payload,
    timeout=30
)

if msg_response.json().get("ok"):
    print(f" Slack summary sent for {repo_name} PR #{pr_number}")
else:
    print(f" Slack message failed: {msg_response.json().get('error')}")
return

# step 2 – upload PDF using new Slack API
try:
    with open(report_path, "rb") as f:
        file_data = f.read()

    filename = os.path.basename(report_path)

    # step 2a – get upload URL
    url_response = requests.post(
        "https://slack.com/api/files.getUploadURLExternal",

```

```

headers={"Authorization": f"Bearer {slack_token}"},
data={
    "filename": filename,
    "length": len(file_data)
},
timeout=30
)

url_data = url_response.json()
if not url_data.get("ok"):
    print(f" File upload failed: {url_data.get('error')}")
    return

upload_url = url_data["upload_url"]
file_id = url_data["file_id"]

# step 2b – upload file content to URL
requests.post(
    upload_url,
    data=file_data,
    headers={"Content-Type": "application/octet-stream"},
    timeout=30
)

# step 2c – complete upload and share to channel
complete_response = requests.post(
    "https://slack.com/api/files.completeUploadExternal",
    headers={
        "Authorization": f"Bearer {slack_token}",
        "Content-Type": "application/json"
    },
    json={
        "files": [{"id": file_id}],
        "channel_id": channel_id,
        "initial_comment": f"Complete audit report for PR #{pr_number}"
    },
    timeout=30
)

```

```
if complete_response.json().get("ok"):
    print(f" PDF report uploaded to Slack")
else:
    print(f" File upload failed: {complete_response.json().get('error')}")

except Exception as e:
    print(f" File upload error: {e}")
```

- **Documentation link**

- https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout